Simulador de Asignación de Memoria y Planificación de Procesos

Sistemas Operativos — UTN FRRe — 2026

Guía de Git y GitHub: ramas, commits, push, pull y Pull Requests

Guía de Git y GitHub para el grupo

Repositorio: https://github.com/javier1793-op/simulador-memoria-procesos

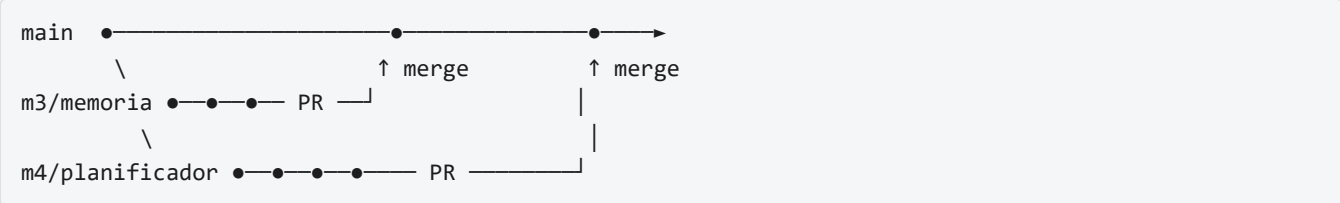
Esta guía es para quienes **nunca usaron Git**. Seguí los pasos en orden y copiá los comandos tal cual. Si algo sale distinto a lo que dice acá, **pará y avisá en el grupo**: no inventes comandos.

---

1. Conceptos en 1 minuto

Palabra	Qué es	Analogía
Repositorio (repo)	La carpeta del proyecto, con todo su historial	Un Google Drive que recuerda cada versión
Commit	Una "foto" guardada de tus cambios, con un mensaje	Guardar una partida
Rama (branch)	Una copia paralela donde trabajás sin tocar la de los demás	Un borrador aparte
main	La rama principal. Tiene la versión que funciona	La versión "oficial"
Push	Subir tus commits a GitHub	Subir el archivo al Drive
Pull	Bajar lo nuevo de GitHub a tu PC	Actualizar el Drive
Pull Request (PR)	Pedirle al líder que revise tu rama y la una a `main`	Entregar el borrador para que lo corrijan
Merge	Unir una rama con otra	Pasar el borrador corregido a la versión oficial

La regla más importante: nadie trabaja directo en `main`. Cada uno trabaja en **su rama**, y Javier aprueba el PR antes de unirla. GitHub no deja hacer merge sin esa aprobación.



---

2. Preparación (UNA sola vez)

2.1 Instalar

1. **Git**: https://git-scm.com/download/win → "Next" en todo (las opciones por defecto están bien).
2. **Python 3.12**: https://www.python.org/downloads/ → tildar **"Add python.exe to PATH"**.
3. **VS Code**: https://code.visualstudio.com + la extensión **Python** (de Microsoft).

2.2 Cuenta de GitHub

1. Crear una cuenta en https://github.com (si no tenés) y **pasarle tu usuario a Javier**.
2. Te va a llegar un mail de invitación al repo → **Accept invitation**.

2.3 Decirle a Git quién sos

Abrí la terminal (en VS Code: menú *Terminal* → *New Terminal*) y escribí, con **tus** datos:

```
git config --global user.name "Tu Nombre"
```

```
git config --global user.email "el-mail-de-tu-cuenta-github@ejemplo.com"
```

2.4 Bajar el proyecto (clonar)

⚠️ **No lo clones dentro de OneDrive, Google Drive ni del Escritorio sincronizado**: la sincronización rompe Git. Usá una carpeta común, como `C:\proyectos`.

```
mkdir C:\proyectos
```

```
cd C:\proyectos
```

```
git clone https://github.com/javier1793-op/simulador-memoria-procesos.git
```

```
cd simulador-memoria-procesos
```

Después abrí esa carpeta en VS Code (*File* → *Open Folder*). La primera vez que hagas `push` se va a abrir el navegador para iniciar sesión en GitHub: aceptá.

Probá que todo funcione:

```
python -m unittest discover -s tests -v
```

Tienen que aparecer tests que **fallan**. Es normal: falta el código que vamos a escribir.

---

3. Cómo nombrar las ramas

Formato: `m<número>/<tema-corto>`

- todo en **minúsculas**
- **sin espacios ni tildes**: las palabras se separan con guion `-`
- empieza con **tu número de miembro**

Miembro	Rama de la primera tarea
M1	`m1/carga`
M2	`m2/proceso`
M3	`m3/memoria`
M4	`m4/planificador`
M5	`m5/admision`
M6	`m6/motor`
M7	`m7/salida`

Para las tareas siguientes, cada una en su propia rama nueva: `m2/casos-de-prueba`, `m7/instalador`, `m1/corrige-validacion`...

❌ Mal: `Rama Juan`, `memoria`, `M3/Memoría`, `prueba` ✅ Bien: `m3/memoria`, `m3/fusion-huecos`

---

4. El flujo de trabajo (cada vez que empezás una tarea)

Paso 1 — Pararte en `main` y bajar lo último

```
git checkout main
```

```
git pull
```

Paso 2 — Crear tu rama

```
git checkout -b m3/memoria
```

(`-b` significa "crearla". Cambiá `m3/memoria` por la tuya.)

Para ver en qué rama estás: `git branch` (la que tiene `*` es la actual). En VS Code también se ve abajo a la izquierda.

Paso 3 — Programar y probar

Editá **sólo los archivos de tu módulo**. Probá seguido:

```
python -m unittest tests.test_memoria -v
```

Paso 4 — Guardar un commit

Ver qué cambiaste:

```
git status
```

Agregar tus archivos (nombrándolos, así no subís cosas de más):

```
git add simulador/memoria.py
```

Hacer el commit con un mensaje:

```
git commit -m "M3: best-fit elige el hueco más chico"
```

Cómo escribir el mensaje: `M<número>: qué hiciste`, en presente y corto.

- ✅ `M3: fusiona huecos libres al liberar`
- ✅ `M1: valida tamaño máximo de 450K`
- ❌ `cambios`, `asdf`, `ya funciona creo`, `final final v2`

Hacé **varios commits chicos** (uno cada vez que algo funciona), no uno gigante al final.

Paso 5 — Subir tu rama a GitHub

La **primera vez** que subís esa rama:

```
git push -u origin m3/memoria
```

Las veces siguientes alcanza con:

```
git push
```

Subí aunque no esté terminado: así queda guardado en GitHub si se te rompe la PC.

Paso 6 — Antes de abrir el PR: traer lo nuevo de `main`

Mientras trabajabas, otros pueden haber unido cosas a `main`. Traelas a tu rama:

```
git pull origin main
```

Volvé a correr tus tests. Si todo anda, `git push`. (Si aparece un **conflicto**, mirá la sección 6.)

Paso 7 — Abrir el Pull Request

1. Entrá al repo en GitHub. Va a aparecer un cartel amarillo **"m3/memoria had recent pushes"** → **Compare & pull request**. (Si no aparece: pestaña **Pull requests** → **New pull request**, *base:* `main`, *compare:* tu rama.)
2. **Título:** `M3: Memoria MVT con Best-Fit`
3. Completá la plantilla que aparece sola (qué hiciste + checklist + link a la tarjeta de Trello).
4. **Create pull request**. A Javier se le pide la revisión automáticamente.
5. Mové tu tarjeta de Trello a **"En revisión"**.

Paso 8 — Si Javier pide cambios

No hace falta abrir otro PR. Corregí en **la misma rama** y subí:

```
git add simulador/memoria.py
```

```
git commit -m "M3: corrige empate de best-fit"
```

```
git push
```

El PR se actualiza solo.

Paso 9 — Cuando Javier hace el merge

Tu trabajo ya está en `main`. Volvé a `main` y bajá lo último antes de la próxima tarea:

```
git checkout main
```

```
git pull
```

Y para la tarea siguiente, **rama nueva** (volvés al Paso 2).

---

5. Resumen para pegar al lado del monitor

```
EMPEZAR TAREA      git checkout main  →  git pull  →  git checkout -b m3/tema
GUARDAR            git status  →  git add <archivo>  →  git commit -m "M3: ..."
SUBIR              git push -u origin m3/tema   (después: git push)
ANTES DEL PR       git pull origin main  →  correr tests  →  git push
PR                 GitHub → Compare & pull request → plantilla → Create
TERMINÓ            git checkout main  →  git pull
```

---

6. Conflictos (no entres en pánico)

Un conflicto pasa cuando vos y otra persona cambiaron **las mismas líneas**. Si cada uno toca sólo su módulo, casi nunca va a pasar.

Si al hacer `git pull origin main` aparece `CONFLICT`:

1. VS Code marca el archivo en rojo. Abrilo: vas a ver bloques así:

   ```
   <<<<<<< HEAD
   tu versión
   =======
   la versión de main
   >>>>>>> main
   ```

2. Hacé clic en **Accept Current** (la tuya), **Accept Incoming** (la de main) o **Accept Both**, según lo que corresponda. Si no sabés cuál, **preguntá en el grupo**.

3. Guardá el archivo y terminá el merge:

   ```
   git add simulador/memoria.py
   ```

```
git commit -m "M3: resuelve conflicto con main"
```

```
git push
```

---

7. SOS: problemas comunes

Me pasa esto	Hacé esto
`fatal: not a git repository`	No estás en la carpeta del proyecto: `cd C:\proyectos\simulador-memoria-procesos`
Hice cambios estando en `main` (sin commit)	`git checkout -b m3/tema` : tus cambios se mueven a la rama nueva
`git push` dice *rejected / fetch first*	`git pull` y después `git push`
`git push` en `main` dice *protected branch*	Está bien, no se puede. Trabajá en tu rama
Quiero descartar los cambios de un archivo (sin commit)	`git restore simulador/memoria.py` ⚠️ se pierden
Me equivoqué en el mensaje del último commit (sin push)	`git commit --amend -m "M3: mensaje correcto"`
Se abrió una pantalla rara al hacer commit (Vim)	Escribí `:q!` y Enter. Usá siempre `-m "mensaje"`
Aparece la carpeta `__pycache__` en `git status`	Ignorala: no la agregues (ya está en `.gitignore`)
Cualquier otra cosa rara	**No uses `--force` ni borres la carpeta.** Sacá captura y mandala al grupo

---

8. Reglas de oro

1. **Nunca** trabajes en `main` .
2. **Una tarea = una rama = un PR.**
3. `git pull` en `main` **antes** de crear una rama.
4. Tocá **sólo los archivos de tu módulo**.
5. Los tests de tu módulo tienen que pasar antes de abrir el PR.
6. Nada de `git push --force` .
7. Ante la duda, **preguntá antes** de ejecutar algo.

---

9. Para Javier: revisar y unir un PR

1. GitHub → **Pull requests** → abrir el PR.
2. Pestaña **Files changed**: revisar que sólo toque su módulo. Se pueden dejar comentarios haciendo clic en el `+` de una línea.
3. Probarlo en tu PC (opcional, pero recomendado):

```
git fetch
```

```
git checkout m3/memoria
```

```
python -m unittest discover -s tests -v
```

4. **Review changes** → *Approve* (o *Request changes* con un comentario de qué corregir).
5. **Squash and merge**: junta todos los commits del PR en uno solo, así `main` queda prolija → **Delete branch**.
6. Mover la tarjeta de Trello a **Hecho**.